

IRAA — 90-Day Build Tracker

How This Works (read once)

This file has two parts:

1. **STATE BLOCK** — a short snapshot of where you are right now. This is the **ONLY** thing you paste into a new LLM chat each day. It always reflects the *current* moment — you overwrite it daily, it never grows.
2. **DAILY CHECKLIST** — the full 90-day plan with checkboxes. This is your personal tracking log, not something you paste into an LLM. Check boxes as you go, jot quick notes if something deviated.

Daily Routine

1. Open this file.
2. Look at today's row in the checklist — that's your task.
3. Update the STATE BLOCK below (takes 60 seconds — see guide under the block).
4. Copy the STATE BLOCK into a new LLM chat with your question/task for the day.
5. Work. Commit to git.
6. Check the box, add a one-line note if anything changed.
7. Update "LAST WORKING STATE" and "TODAY'S GOAL" in the STATE BLOCK for tomorrow.

● STATE BLOCK (update this daily — paste this whole block into new LLM chats)

```
PROJECT: IRAA — Local Voice Assistant
CURRENT DAY: 1 / 90
PHASE: 1 — Core Voice Loop (Days 1-14)
STACK (LOCKED, do not change): Python, FastAPI, SQLite, faster-whisper, Ollama
(Llama 3/Mistral/Qwen), sentence-transformers, ChromaDB/FAISS, Piper TTS, Docker,
GitHub

COMPLETED SO FAR:
- (nothing yet — Day 1 not started)

LAST WORKING STATE:
- No code written yet.
```

```
TODAY'S GOAL (from checklist):
- Day 1: Create GitHub repo "IRAA", set up folder structure (backend/, voice/, llm/,
tts/, stt/, core/, tests/), install Python env + FastAPI + faster-whisper +
requests.

BLOCKERS / DEVIATIONS FROM PLAN:
- None yet.

NEXT 3 DAYS PREVIEW:
- Day 2: Microphone input, save .wav locally
- Day 3: Whisper STT integration
- Day 4: Ollama setup, run LLM locally
```

How to update this block each day: move completed items from "TODAY'S GOAL" into "COMPLETED SO FAR" (one line, terse — e.g. "Day 3: STT working, whisper.py transcribes wav to text"), update "LAST WORKING STATE" to describe what currently runs end-to-end, pull tomorrow's task into "TODAY'S GOAL," and shift the preview window forward. If you skip a day or change the plan, log it in BLOCKERS so the LLM knows the plan and reality have diverged.

Daily Checklist

Phase 1 (Days 1-14): Core Voice Loop — Goal: speak → hear → LLM responds → speaks back

Day	Task	Output	Done	Notes
1	GitHub repo "IRAA" + folder structure (backend/, voice/, llm/, tts/, stt/, core/, tests/) + install Python env, FastAPI, faster-whisper, requests	Repo initialized	[]	
2	Mic input, record audio, save .wav locally	<code>record.py</code> working	[]	
3	Integrate Whisper STT, audio → text	"Hello" spoken → text output	[]	
4	Setup Ollama locally, run Llama 3/Mistral	Prompt → response in terminal	[]	

Day	Task	Output	Done	Notes
5	Connect STT → LLM pipeline	First working voice chatbot (text only)	☐	
6	Add TTS (Piper), LLM response → voice	Full loop working	☐	
7	Refactor into modules: stt.py, llm.py, tts.py	Clean structure	☐	
8	Add conversation history (in-memory list)		☐	
9	Improve prompt system: system prompt + IRAA personality		☐	
10	Error handling: silence, noise, empty input		☐	
11	Improve latency, stream responses if possible		☐	
12	CLI interface: "Press Enter to talk"		☐	
13	Basic file logging system		☐	
14	MILESTONE 1: record demo video, speak → IRAA responds	✓ Working voice assistant v0.1	☐	

Phase 2 (Days 15-35): Memory + Tools — Goal: IRAA remembers and does things

Day	Task	Output	Done	Notes
15-16	Learn embeddings (sentence-transformers), setup Chroma		☐	
17	Build memory store: save conversations		☐	
18	Retrieval system: query memory → return context		☐	
19	Connect memory to LLM prompt		☐	
20	Test: "what did I say yesterday?"		☐	
21-22	Tool system design (open_app, search_files, etc.)		☐	
23-24	Implement file search tool		☐	
25-26	Implement system commands (open VS Code, open browser)		☐	
27	LLM decides tool usage (basic function calling)		☐	

Day	Task	Output	Done	Notes
28-30	Improve tool router logic		☐	
31-32	Structured JSON responses from LLM		☐	
33-35	MILESTONE 2: memory + tools working demo		☐	

Phase 3 (Days 36-60): Real Assistant — Goal: useful in daily life

Day	Task	Output	Done	Notes
36-38	Calendar integration (local or Google API)		☐	
39-41	File system assistant: search / open / summarize files		☐	
42-45	Web search integration (DuckDuckGo API or scraping)		☐	
46-48	Email summarization (mock or real Gmail API optional)		☐	
49-52	Improve memory: long-term + short-term split		☐	
53-55	Context summarization system		☐	
56-60	MILESTONE 3: "Daily assistant mode" — morning briefing, tasks, reminders		☐	

Phase 4 (Days 61-80): Productization — Goal: interview-worthy

Day	Task	Output	Done	Notes
61-63	FastAPI backend separation		☐	
64-66	WebSocket streaming voice		☐	
67-69	Docker setup		☐	
70-72	CI/CD (GitHub Actions)		☐	

Day	Task	Output	Done	Notes
73-75	Logging + monitoring		☐	
76-78	Config system (.env, secrets)		☐	
79-80	MILESTONE 4: fully deployable system		☐	

Phase 5 (Days 81-90): Interview Prep + Polish

Day	Task	Output	Done	Notes
81-83	Architecture diagrams, system design explanation		☐	
84-86	Resume rewrite — IRAA as flagship project		☐	
87-88	Mock interviews: backend, system design, project explanation		☐	
89	Deploy demo video + GitHub cleanup		☐	
90	FINAL DEMO: full assistant flow recorded, apply to jobs		☐	

Ground Rules (keep visible)

1. No new tech exploration after Week 2.
2. Every day produces at least ONE tangible output (code, commit, or deployed change).
3. No skipping "boring" tasks (logging, structure, refactoring).
4. If stuck >2 hours → simplify, don't switch stack.
5. Git commit daily, minimum.